

Лабораторная работа 6 **УПРАВЛЕНИЕ ОТОБРАЖЕНИЕМ ДАННЫХ В ПРИЛОЖЕНИИ**

Цель работы: ознакомиться с функциональными аспектами представления графических данных содержащихся в BLOB-полях, а также механизмами сортировки, как на основе SQL запроса, так и средствами ADO-компонент; изучить механизмы настройки параметров отображения данных в компоненте TDBGrid; разработать приложение, учитывающее различные аспекты отображения данных.

ПОРЯДОК ВЫПОЛНЕНИЯ РАБОТЫ

1. Ознакомиться с теоретическими сведениями.
2. Организовать отображение/назначение изображений товаров в приложении.
3. Сделать сортировку записей с привязкой к событию **OnTitleClick** на основе **SQL**-кода для складских запасов.
4. Сделать сортировку записей операций на основе свойства **sort** **ADO**-компонент по событию **OnTitleClick**.
5. Установить подсветку записей в компоненте **TDBGrid**.
6. Организовать изменение параметры подсветки записей.
7. Ответить на контрольные вопросы.

ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

Тип данных BLOB применяется для хранения любых бинарных данных, но чаще всего используется для хранения изображений, документов или других мультимедийных объектов.

6.1 РАБОТА С ИЗОБРАЖЕНИЯМИ В ПОЛЯХ BLOB

В качестве примера работы с изображениями в полях BLOB рассмотрим хранение фотографий товаров. Для этой цели в таблице **goods_catalog** было создано поле **gcPhoto**.

Разместим на форму **Form4** (рис 6.1) панель на которую поместите компонент **TDBImage** для отображения в нем фотографий. Для панели сделаем выравнивание по правому краю (свойство **Align** со значением **alRight**).

У компонентов **TImage** и **TDBImage** имеется ряд свойств определяющих параметры отображения. Так, свойство **Stretch** в активном состоянии говорит о том, что отображаемый рисунок/фотография будет подогнан до размеров области компонента. Свойство **Proportional** указывает на

выполнение пропорционального изменения размера. Свойство **Center** означает позиционирование отображаемого рисунка по центру области компонента.

Компоненту **TDBImage** установим следующие свойства как указано в таблице 6.1.

Таблица 6.1

Свойства компонента **TDBImage** для отображения фотографии товаров

Свойство	Значение
Align	alTop
AlignWithMargins	True
Center	True
DataSource	DS_T_goods_catalog
DataField	gcPhoto
Name	Goods_Image
Stretch	True
Propotional	True

После настройки области отображения фотографий на панель **PanelToInputValues** разместим две кнопки которым дадим следующие имена **B_SetImage** – **Назначить картинку** и **B_ClearImage** – **Очистить картинку**. Для того чтобы можно было выбрать картинку на форма необходимо разместить компонент **OpenPictureDialog** (имя по умолчанию **OpenPictureDialog1**). После размещения компонентов форма примет следующий вид (рис 6.1):

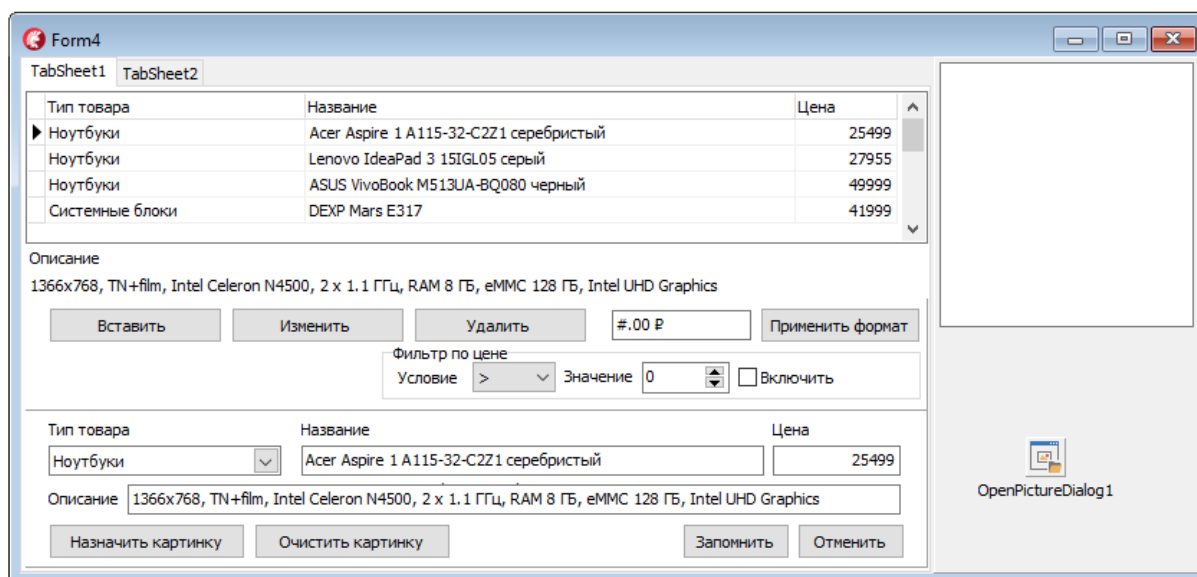


Рис. 6.1. Добавление изображения на форму

Теперь пропишем обработчик события **OnClick** кнопки **B_SetImage** для добавления картинки в базу данных и её отображения в компоненте **Goods_Image**:

```

procedure TForm4.B_SetImageClick(Sender: TObject);
begin
    // Проверка запустилось ли диалоговое окно
    if OpenPictureDialog1.Execute then
        // в случае если пользователь нажал на кнопку Ok
        // в диалоговом окне проверяем существует ли выбранный файл
        if FileExists(OpenPictureDialog1.FileName) then
            begin
                // загружаем выбранный файл в компонент Goods_Image
                Goods_Image.Picture.LoadFromFile(OpenPictureDialog1.FileName);
                // загружаем выбранный файл в поле gcPhoto
                DataModule2.T_goods_cataloggcPhoto.LoadFromFile
                    (OpenPictureDialog1.FileName);
            end;
        end;
end;

```

Для удаления картинки из поля **gcPhoto** и из компонента **Goods_Image** будет использован обработчик события **OnClick** кнопки **B_ClearImage**:

```

procedure TForm4.B_ClearImageClick(Sender: TObject);
begin
    // вызываем метод очистки поля
    DataModule2.T_goods_cataloggcPhoto.Clear;
    // назначаем значение nil для картинки
    Goods_Image.Picture := nil;
end;

```

В связи с тем, что визуализация данных поля **gcPhoto** осуществляется с использованием отдельного компонента, то в режиме добавления записи нужно предусмотреть очистку картинки в компоненте **Goods_Image**.

Для этого в событие **OnClick** кнопки **InsertButton** необходимо добавить код: `Goods_Image.Picture := nil;`

Измененный код обработчика событий будет выглядеть следующим образом:

```

procedure TForm4.InsertButtonClick(Sender: TObject);
begin
    if DataModule2.T_goods_catalog.State = dsBrowse then
        begin
            // очистить отображаемую картинку
            Goods_Image.Picture := nil;
            // Отключить возможность изменения курсора НД
            // в компоненте DBGrid1
            DBGrid1.Enabled := False;
            // Сделать невидимой панель с кнопками Вставить, Изменить,
            // Удалить
            InsertEditDeletePanel.Visible := False;
            // Сделать видимой панель для ввода значения в запись

```

```

PanelToInputValues.Visible := True;
// Перевести НД в режим добавления записи, используя
// метод Append
DataModule2.T_goods_catalog.Append;
end;
end;

```

После настройки необходимых обработчиков запустим приложение и убедимся что функционал по взаимодействию с полем **gcPhoto** работает нормально (рис 6.2):

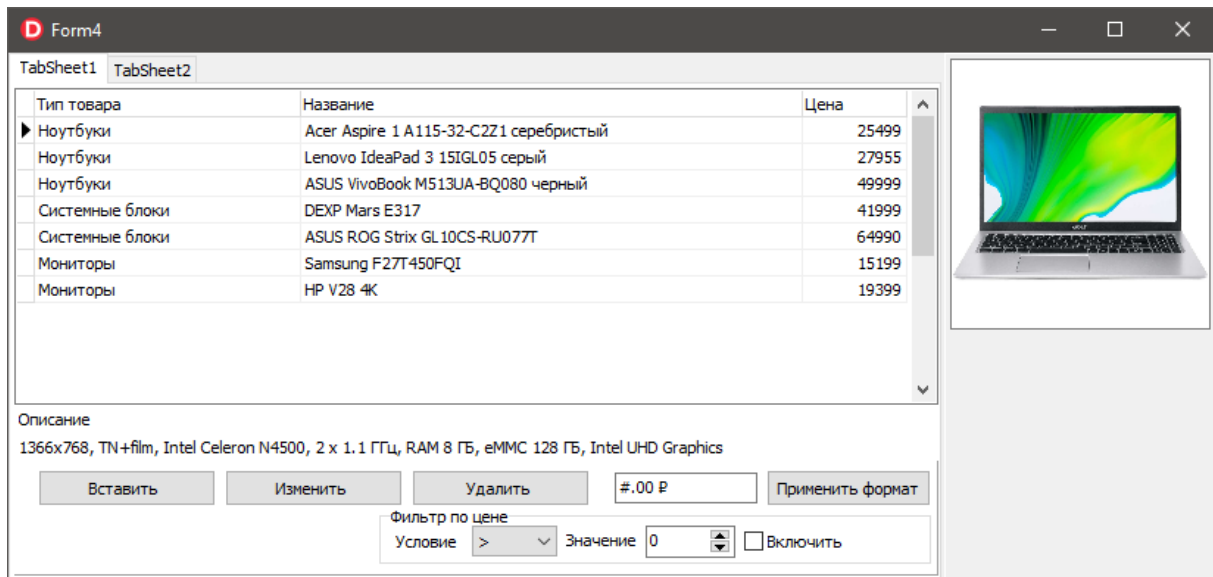


Рис. 6.2. Пример отображения фотографий товаров

Теперь необходимо добавить изображения к товарам в каталог товаров через кнопку **Изменить**.

6.2 ПОНЯТИЕ СТОЛБЦОВ В КОМПОНЕНТЕ TDBGrid

Компонент **TDBGrid** используется для показа содержимого записей НД в табличном формате, когда строки соответствуют записям НД, а столбцы — полям записи. Свойство **DataSource** компонента **TDBGrid** содержит имя компонента **TDataSource**, который ссылается на соответствующий НД — компоненты **TADOTable** или **TADOQuery**.

Рассмотрим основные моменты, связанные с настройкой визуальных характеристик отображаемых полей в компоненте **TDBGrid**. Поскольку все поля в этом компоненте задаются в виде столбцов, то покажем, что представляют собой столбцы, и опишем их основные свойства.

Для определения состава столбцов в компоненте **TDBGrid** можно использовать редактор столбцов **Column Editor**. В этом случае учитываются только столбцы, созданные в редакторе столбцов компонента **TDBGrid**, и принимаются во внимание только их характеристики (в част-

ности, порядок следования). Фактически в редакторе столбцов определяются постоянные столбцы. Ранее мы уже определяли набор отображаемых столбцов и использовали часть свойств для настройки отображения.

Если редактор столбцов не применялся, то берутся поля, объявленные при помощи редактора полей НД (компоненты **TField**). Столбцы компонента **TDBGrid**, основанные на использовании компонентов **TField**, называются *динамическими столбцами*. При этом вид столбцов определяется соответствующими характеристиками компонентов **TField**, а порядок следования столбцов – порядком их определений. В таком случае для изменения заголовков столбцов в редакторе полей следует изменить значения свойств **DisplayLabel** для каждого поля.

В случае, когда компоненты **TField** для НД не создавались, порядок следования полей и их характеристики соответствуют тем, что были заданы при определении структуры данной таблицы базы данных в момент ее создания.

Многие свойства компонента **TDBGrid** можно переустанавливать во время выполнения приложения, что предоставляет разработчикам большую гибкость при реализации интерфейса.

Таблица 6.2

Основные свойства столбцов компонента **TDBGrid**

Свойство	Назначение
Alignment	Определяет выравнивание значений в столбце. По умолчанию соответствует свойству Field.Alignment
Color	Определяет цвет фона столбца. По умолчанию соответствует свойству TDBGrid.Color
FieldName	Определяет поле таблицы базы данных, ассоциированное с данным постоянным столбцом
ReadOnly	Определяет возможность редактирования столбца из компонента TDBGrid (значение True) или невозможность этого (значение False по умолчанию)
Width	Определяет ширину столбца в пикселях. По умолчанию значение извлекается из свойства Field.DisplayWidth , где ширина задается в символах
Font	Определяет тип, размер и цвет шрифта для вывода значений в столбце. По умолчанию соответствуют свойству TDBGrid.Font
Title.Alignment	Определяет выравнивание заголовка. По умолчанию выполняет выравнивание влево
Title.Caption	Определяет надпись заголовка. По умолчанию соответствует свойству Field.DisplayLabel или имени поля таблицы базы данных
Title.Font	Определяет тип, размер и цвет шрифта заголовка. По умолчанию соответствует свойству TDBGrid.TitleFont

Если необходимо изменить свойства постоянного столбца компонента **TDBGrid**, то к нему можно обратиться как к **DBGrid1.Columns.Items[i]** или как к **DBGrid1.Columns[i]**, где **i** принадлежит диапазону **[0 .. DBGrid1.Columns.Count – 1]**.

Рассмотрим пример реализации изменения сортировки наборов данных на основе заголовков столбцов компонента **TDBGrid**.

6.3 ИЗМЕНЕНИЕ СОРТИРОВКИ ПОЛЕЙ ВО ВРЕМЯ ВЫПОЛНЕНИЯ ПРИЛОЖЕНИЯ НА ОСНОВЕ СОБЫТИЯ **ONTITLECLICK**

Часто при просмотре данных в компоненте **DBGrid** появляется необходимость изменить сортировку НД по какому-нибудь столбцу. Для реализации механизма сортировки можно использовать два подхода:

- на основе изменение SQL-кода запроса компонента **ADOQuery**;
- использование свойства **Sort** ADO-компонентов.

Наиболее оптимальным решением является написание обработчика события **OnTitleClick**, которое возникает, когда мы щелкаем мышью по заголовкам столбцов.

Сначала рассмотрим реализацию сортировки на основе изменения SQL-кода на примере сортировки складских запасов (Отображение данных в **DBGrid1** на вкладке «Состояние склада»).

Поскольку изменение отображаемого набора данных **Q_v_storage** происходит с учетом фильтрации, то необходимо выполнять изменение SQL-кода фильтра с учетом параметров сортировки.

В SQL-запросах на выборку сортировка определяется при помощи опциональной части – оператора **ORDER BY**. Для задания сортировки необходимо указать имя поля (или его номер) и тип сортировки. По умолчанию данные сортируются по возрастанию, однако с помощью оператора **DESC** можно задать сортировку по убыванию. При желании для сортировки можно использовать и выражение, однако в нашем примере мы ограничимся именем столбца.

Для реализации механизма сортировки на основе заголовков столбцов нужно объявить глобальные переменные:

- **CSort_st** целочисленного типа, которая будет показывать, по какому столбцу выполнялась сортировка в прошлый раз,
- **OrderBy_field** типа *string* которая будет содержать конструкцию сортировки (имя поля и тип сортировки – по возрастанию или убыванию).

В блоке **var** основной формы (**Form1**) добавим объявление следующего вида:

```
CSort_st: Integer = -1; // Инициализировать значение -1, так как
                        // при запуске программы не было сортировки
OrderBy_field: string; // параметр сортировки для SQL-запроса
```

После объявления глобальных переменных, необходимо изменить код обработчика события **OnChange** для компонента **E_Filter** (реализует фильтрацию на основе SQL-кода) следующим образом:

```
procedure TForm1.E_FilterChange(Sender: TObject);
var
  baseSQL, filter_s: string;
  orderBy_s: string; // Модификация (объявление переменной)
begin
  DataModule2.Q_v_storage.DisableControls;
  DataModule2.Q_v_storage.Active := false;
  baseSQL := 'SELECT * FROM view_storage ';
  filter_s := '';
  orderBy_s := ''; // Модификация (задание пустого значения)
  if length(E_Filter.Text) > 0 then
    filter_s := 'WHERE tgName LIKE ' + QuotedStr('%' + E_Filter.Text +
      '%') + ' OR gcName LIKE ' + QuotedStr('%' + E_Filter.Text + '%');
  // Модификация (проверка указано ли поле сортировки)
  if length(OrderBy_field) > 0 then
    // задание строки с параметром сортировки,
    // если дина значения OrderBy_field больше 0
    orderBy_s := ' ORDER BY ' + OrderBy_field;
  DataModule2.Q_v_storage.SQL.Text := baseSQL + filter_s + orderBy_s;
  DataModule2.Q_v_storage.Active := true;
  DataModule2.Q_v_storage.EnableControls;
end;
```

Следует обратить внимание, что сортировка возможна только по полям с данными. Поля, значения которых генерируются во время работы приложения (поля выбора или вычисляемые поля), не могут быть использованы для сортировки. Если в ваших наборах данных будут присутствовать подобные поля, то следует выполнять предварительную проверку следующим образом:

```
if.ADOQuery.FieldByName(Column.FieldName).FieldKind = fkData then
```

В нашем наборе столбцов из НД **Q_v_storage** такие поля отсутствуют, поэтому данную проверку проводить не будем.

Для того чтобы понимать в каком направлении ведется сортировка предлагается у заголовков столбцов делать зеленый цвет (*clGreen*) для сортировки по возрастанию и синий (*clBlue*) в случае сортировки по убыванию. При этом заголовок у столбца, по которому идет сортировка, будет выделяться жирным шрифтом. Обработчик события **OnTitleClick** для **DBGrid1** в нашем примере будет выглядеть следующим образом:

```
procedure TForm1.DBGrid1TitleClick(Column: TColumn);
var
  sorttype: string;
begin
  // Сбросим параметры для колонки, с которой работали в прошлый раз
```

```

if CSort_st >= 0 then
begin
    DBGrid1.Columns[CSort_st].Title.Font.Color := clBlack;
    DBGrid1.Columns[CSort_st].Title.Font.Style :=
        Column.Title.Font.Style - [fsBold];
end;
// Установим параметр сортировки как пустое значение
// (соответствует сортировке по возрастанию ASC)
sorttype := '';
// Запомним индекс сортируемого столбца в глобальную переменную
CSort_st := Column.Index;
// Проверим, не выполнялась ли сортировка по убыванию
// для данного поля
if OrderBy_field <> Column.FieldName + ' DESC' then
begin
    // Если проверка не выполнялась, то сверим, сортировали ли
    // по этому полю или нет
    if OrderBy_field = Column.FieldName then
        // Если сортировка по данному полю уже была, то поменяем
        // тип сортировки
        sorttype := ' DESC';
    // Зададим стили шрифта заголовка для отображения вида сортировки
    Column.Title.Font.Style := Column.Title.Font.Style + [fsBold];
    if sorttype = '' then
        Column.Title.Font.Color := clGreen // По возрастанию
    else
        Column.Title.Font.Color := clBlue; // По убыванию
    // Установим сортировку для поля с заданным параметром
    OrderBy_field := Column.FieldName + sorttype;
end
else
    // Если в прошлый раз использовалась сортировка по убыванию,
    // то отключим сортировку
    OrderBy_field := '';
// поскольку нужно обновить запрос то воспользуемся вызовом
// кода прописанном на фильтре
E_FilterChange(self);
end;

```

Для проверки как будет работать сортировка на основе изменения SQL-кода можно запустить приложение.

Теперь рассмотрим, как будет реализовываться сортировка с использованием свойства **sort** ADO-компонентов, на примере списка операций (набор данных **Q_v_goods_operations**). Также как и в случае с предыдущим видом сортировки нам потребуется переменная для хранения номера столбца – **CSort_op**. Объявим её в блоке **var** формы **Form1**:

```

CSort_op: Integer = -1; // Инициализировать значение -1, так как
                        // при запуске программы не было сортировки

```


Поскольку отображения перечня операций происходит в **DBGrid2** то для него зададим обработчик события **OnTitleClick** следующим образом:

```
procedure TForm1.DBGrid2TitleClick(Column: TColumn);
var
  sorttype: string;
begin
  // Сбросим параметры для колонки, с которой работали в прошлый раз
  if CSort_op >= 0 then
  begin
    DBGrid2.Columns[CSort_op].Title.Font.Color := clBlack;
    DBGrid2.Columns[CSort_op].Title.Font.Style :=
      Column.Title.Font.Style - [fsBold];
  end;
  // Установим параметр сортировки как пустое значение
  // (соответствует сортировке по возрастанию ASC)
  sorttype := '';
  // Запомним индекс сортируемого столбца
  CSort_op := Column.Index;
  // Проверим, не выполнялась ли сортировка по убыванию
  // для данного поля
  if DataModule2.Q_v_goods_operations.Sort <> Column.FieldName +
    ' DESC' then
  begin
    // Если проверка не выполнялась, то сверим, сортировали ли
    // по этому полю или нет
    if DataModule2.Q_v_goods_operations.Sort = Column.FieldName then
      // Если сортировка по данному полю уже была, то поменяем
      // тип сортировки
      sorttype := ' DESC';
    // Зададим стили шрифта заголовка для отображения вида сортировки
    Column.Title.Font.Style := Column.Title.Font.Style + [fsBold];
    if sorttype = '' then
      Column.Title.Font.Color := clGreen // По возрастанию
    else
      Column.Title.Font.Color := clBlue; // По убыванию
    // Установить сортировку для поля с заданным параметром
    DataModule2.Q_v_goods_operations.Sort :=
      Column.FieldName + sorttype;
  end
  else
    // Если в прошлый раз использовалась сортировка по убыванию,
    // то отключим сортировку
    DataModule2.Q_v_goods_operations.Sort := '';
end;
```

В представленном исходном коде стоит обратить внимание на пробелы, которые добавляются к строковым переменным, например:

```
sorttype := ' DESC';.
```

Описанные механизмы сортировки работают используя один и тот же принцип – выясняют поле, по которому необходимо выполнить сортировку в момент работы программы. В связи с этим данные подходы можно использовать как универсальное средство во время разработки разнообразных программных систем, использующих отображение данных с возможностью сортировки.

6.4 ПОДСВЕТКА ЗАПИСЕЙ В КОМПОНЕНТЕ DBGRID

Рассмотрим средства выделения (цветом, изменением шрифта и т. д.) какого-либо столбца в компоненте **TDBGrid**. Предварительно отметим, что на характеристики представления значений в полях компонента **TDBGrid** могут оказывать влияние такие свойства компонентов **TField**, как **DisplayFormat** и **EditFormat**, которые позволяют задавать формат поля при его показе и редактировании соответственно. Эти свойства актуальны для всех компонентов, визуализирующих значение поля (полей), а не только для столбцов компонента **TDBGrid**.

Средства настройки отображения с изменением свойств столбцов носят безусловный характер, т. е. они позволяют изменять характеристики показа всех значений в конкретном столбце. Однако может потребоваться выделение не целого столбца, а определенных полей столбца, отвечающих какому-либо условию. Например, для бухгалтерских и складских программ часто принято выделять красным цветом отрицательные остатки или остатки, находящиеся ниже установленного лимита.

В нашем случае мы будем рассматривать подсветку складских запасов значение которых меньше чем значение определенное пользователем.

Способ прорисовки данных в компоненте **TDBGrid**: стандартный или по особому сценарию – определяется свойством **DefaultDrawing** имеющим тип **Boolean**.

Если прорисовка управляется со стороны самого приложения, то алгоритм прорисовки должен содержаться в обработчиках событий **OnDrawColumnCell** или **OnDrawDataCell** (этот обработчик введен для обеспечения совместимости с ранними версиями **Delphi**).

При автоматической прорисовке (свойство **DefaultDrawing** установлено в значение **True**) метод **Paint** использует цвет и шрифт свойства **Canvas** для прорисовки полей.

Событие **OnDrawColumnCell**:

```
TDrawColumnCellEvent = procedure(Sender: TObject; const Rect: TRect;  
DataCol: Integer; Column: TColumn; State: TGridDrawState) of object;
```

имеет следующие параметры:

- **Rect: TRect** определяет область поля, где происходит прорисовка.
- **DataCol: Integer** показывает индекс текущего столбца;

- **Column: TColumn** задает текущий столбец;
- **State: TGridDrawState** характеризует состояние поля.

Для задания прямоугольной области используется тип **TRect**. Он имеет следующую структуру:

```
TRect = record
  case Integer of
    0: (Left, Top, Right, Bottom: Integer);
    1: (TopLeft, BottomRight: TPoint);
  end;
end;
```

Координаты указываются или как четыре целых числа, содержащих границы прямоугольника в пиксельном исчислении, или как два значения типа **TPoint**, соответствующих левому верхнему и правому нижнему углам прямоугольника:

```
TPoint = record
  X: LongInt;
  Y: LongInt;
end;
```

Состояние поля характеризует тип **TGridDrawState** который представляет собой набор допустимых значений (возможных состояний поля):

```
TGridDrawState = set of (gdSelected, gdFocused, gdFixed);
```

Если поле находится в состоянии **gdFocused**, т. е. является текущим, то в обработчике события **OnDrawColumnCell** происходит прорисовка инверсной полосы.

Для вывода полей по умолчанию используется метод

```
procedure DefaultDrawColumnCell(const Rect: TRect; DataCol:
  Integer; Column: TColumn; State: TGridDrawState);
```

Отображение как целых строк так и отдельных полей можно изменять, используя информацию содержащуюся в передаваемых в обработчик события **OnDrawColumnCell** параметрах.

Рассмотрим механизм настройки подсветки записей на вкладке «Состояние склада». Для начала расширим панель по высоте и разместим компоненты **TLabel** и **TSpinEdit** (Имя по умолчанию **SpinEdit1**). В компоненте **SpinEdit1** установим следующие свойства: **MinValue** в значение **1**, **MaxValue** в значение **1000**, **Value** значение **1**. В свойство **Caption** компонента **Tlabel** пропишем поясняющий текст. Пример расположения элементов и результата их настройки приведен на рисунке 6.3

Нам необходимо вывести белым шрифтом на красном фоне те строки компонента НД **Q_v_storage**, у которых поле **sCount** содержит значение меньше либо равное значению указанного в **SpinEdit1**.

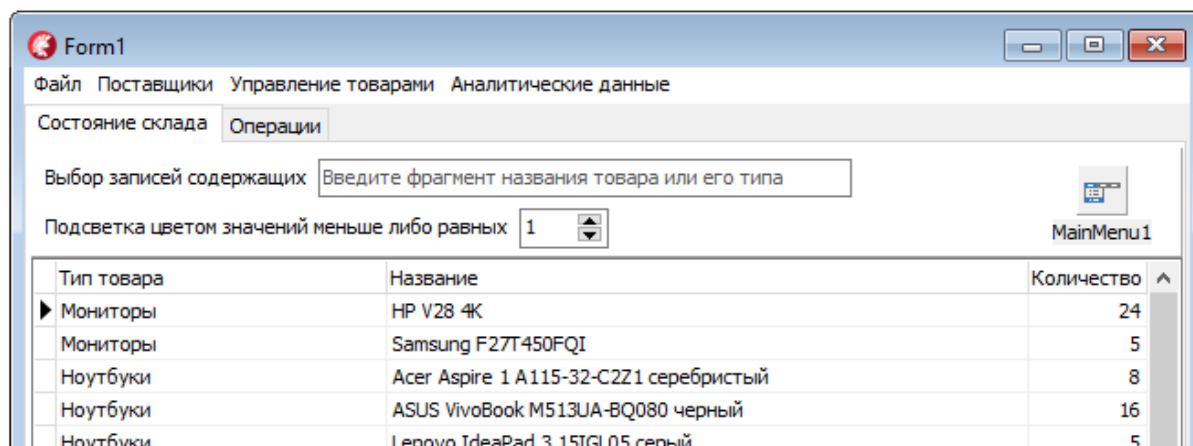


Рис 6.3. Пример вида компонентов после размещения и настройки

В случае программного изменения цвета только согласно оговоренного условия запись имеющая выделение (находится в состоянии **gdSelected**) не будет отличаться от остальных. Для того чтобы визуальное можно было определить такую запись для нее будем устанавливать бордовый цвет фона.

С учетом этого код обработчика события **OnDrawColumnCell** для **DBGrid1**, в котором происходит отображение складских запасов, примет следующий вид:

```
procedure TForm1.DBGrid1DrawColumnCell(Sender: TObject; const Rect:
  TRect; DataCol: Integer; Column: TColumn; State: TGridDrawState);
begin
  with DBGrid1.Canvas do
  begin
    // Проверим, не меньше ли значение поля sCount
    // чем значение установленное в SpinEdit1
    if (DataModule2.Q_v_storagesCount.value <= SpinEdit1.value) then
    // Если условие выполняется, то зададим формат вывода
    begin
      // Установим красный цвет фона
      Brush.Color := clRed;
      // Проверим если строка (запись) выбрана
      if (gdSelected in State) then
        // то установим бордовый цвет фона
        Brush.Color := clMaroon;
      // Установим белого цвет шрифта
      Font.Color := clWhite;
    end;
  end; // With
  // Выполним отрисовку поля стандартным методом
  DBGrid1.DefaultDrawColumnCell(Rect, DataCol, Column, State);
end;
```

Для того чтобы фокусировка (выделение) была по всей строке необходимо чтобы была активна опция **dgRowSelect** в компоненте **TDBGrid**.

Организовать полную перерисовку компонента **DBGrid1** при изменении значения в **SpinEdit1** можно при помощи метода **Invalidate** прописав соответствующий код в событии **OnChange**:

```
procedure TForm1.SpinEdit1Change(Sender: TObject);
begin
    DBGrid1.Invalidate;
end;
```

Теперь можно запустить приложение и посмотреть на результат работы (Рис. 6.4).

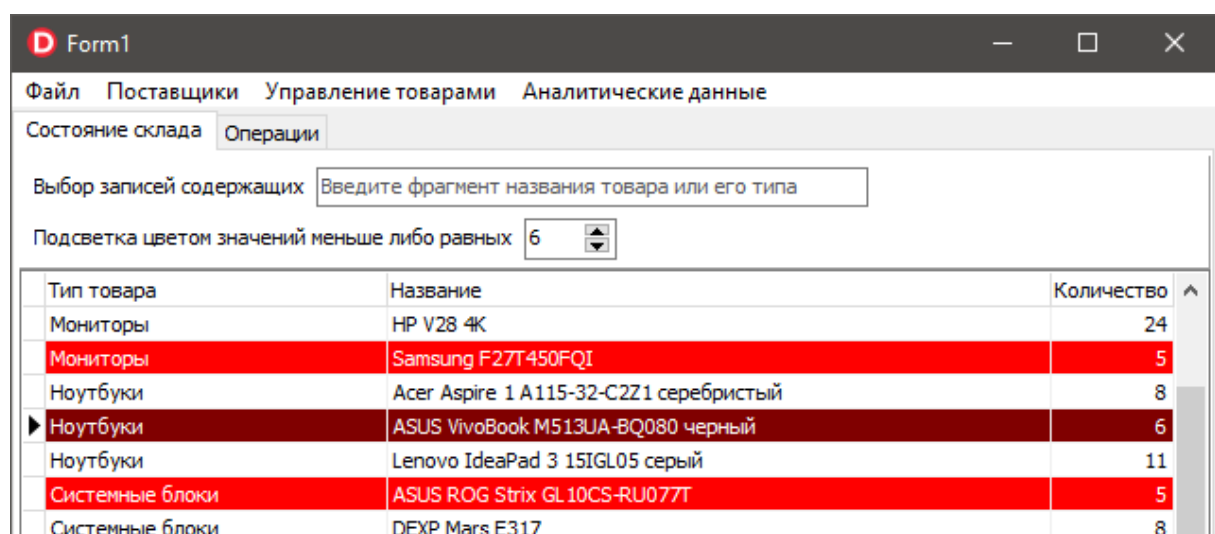


Рис 6.4. Пример работы условной подсветки записей

Если нужно подсвечивать не всю запись, а только поле, то можно воспользоваться кодом представленным ниже:

```
procedure TForm1.DBGrid1DrawColumnCell(Sender: TObject; const Rect:
    TRect; DataCol: Integer; Column: TColumn; State: TGridDrawState);
begin
    with DBGrid1.Canvas do
    begin
        // Проверяем имя поля
        if (Column.FieldName = 'sCount') then
            // Проверим, не меньше ли значение поля sCount
            // чем значение установленное в SpinEdit1
            if (DataModule2.Q_v_storagesCount.value <= SpinEdit1.value) then
                // Если условие выполняется, то зададим формат вывода
                begin
                    // Установим красный цвет фона
                    Brush.Color := clRed;
                    // Проверим если строка (запись) выбрана
                    if (gdSelected in State) then
```

```

        // то установим бордовый цвет фона
        Brush.Color := clMaroon;
        // Установим белого цвет шрифта
        Font.Color := clWhite;
    end;
end; // With
// Выполним отрисовку поля стандартным методом
DBGrid1.DefaultDrawColumnCell(Rect, DataCol, Column, State);
end;

```

Пример работы такого механизма подсветки показан на рисунке 6.5.

Тип товара	Название	Количество
Мониторы	HP V28 4K	24
Мониторы	Samsung F27T450FQI	5
Ноутбуки	Acer Aspire 1 A115-32-C2Z1 серебристый	8
Ноутбуки	ASUS VivoBook M513UA-BQ080 черный	6
Ноутбуки	Lenovo IdeaPad 3 15IGL05 серый	11
Системные блоки	ASUS ROG Strix GL10CS-RU077T	5
Системные блоки	DFXP Mars F317	8

Рис 6.5 Пример работы условной подсветки поля

Фиксированная подсветка (подсветка определенная разработчиком) подходит для большинства задач, однако может возникнуть ситуация когда стандартная цветовая схема не подходит. Рассмотрим возможности по настройке параметров настройки цвета и шрифта во время работы приложения.

6.5 ДИНАМИЧЕСКАЯ НАСТРОЙКА ПАРАМЕТРОВ ШРИФТА И ЦВЕТА ПРИ ОРГАНИЗАЦИИ ПОДСВЕТКИ ЗАПИСЕЙ В DBGRID

Рассмотрим пример реализации динамического изменения значений свойств подсветки записей. Для этого разместим на форме **Form1** невизуальные компоненты, обеспечивающие диалоги выбора шрифта **TFontDialog** и цвета **TColorDialog** (они находятся на вкладке **Dialogs** палитры компонентов среды **Delphi**), а также два набора компонентов которые будут отвечать за настройку записей (без выделения и с выделением при помощи курсором).

Для каждого набора понадобятся следующие компоненты: **TGroupBox** – для группировки элементов и пояснения группы, **TPanel** для

имитации вида подсвеченной записи, а также две кнопки для выбора цвета фона и шрифта.

Сначала зададим параметры для компонентов настройки подсветки записей по умолчанию (т.е. записи, не выделенной курсором). Кнопки назовем **B_NA_ColorSel** и **B_NA_FontSel**, в их свойство **Caption** пропишем **Выбор цвета** и **Выбор шрифта** соответственно. Компонент **TPanel**, для демонстрации оформления назовем **p_NA_record** и зададим следующие свойства (Табл. 6.3)

Таблица 6.3

Свойства компонента **p_NA_record** для отображения фотографии товаров

Свойство	Значение
BevelInner	bvLowered
Caption	текст записи
Color	clRed
Font.Color	clWhite

Следующий набор компонент, предназначенный для настройки параметров отображения выделенной записи (активной записи – в состоянии **gdSelected**) будет настроен аналогичным образом. Имена компонентов **TButton** будут **B_A_ColorSel** и **B_A_FontSel**, а для **TPanel** – **p_A_record**. Отличие в настройках свойств панели **p_A_record** от **p_NA_record** заключается в цвете – свойство **Color** будет иметь значение **clMaroon**.

После настройки компонентов изменим значение свойства **Caption** у компонент **TGroupBox** для пояснения их назначения. Пример размещения с заданием свойств компонентов приведен на рисунке 6.6.

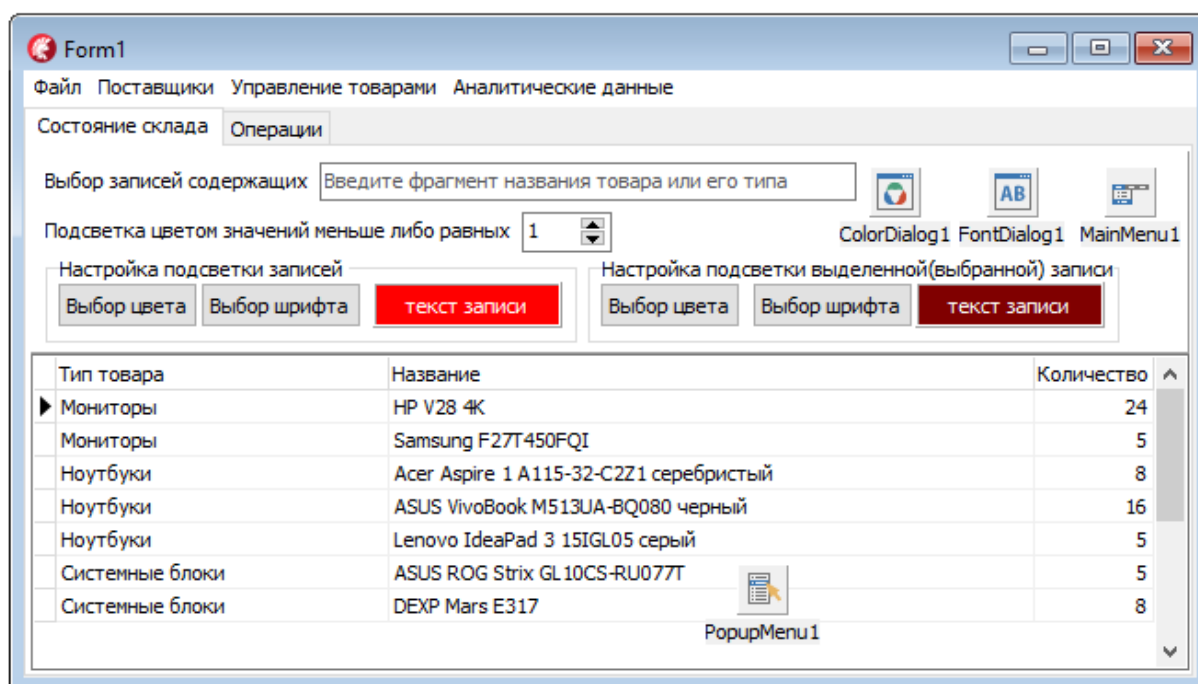


Рис 6.6 Пример размещения компонентов с настройкой параметров

Для настройки двух состояний будут использоваться одни и те же диалоговые окна **ColorDialog1** и **FontDialog1**. В связи с этим для каждого действия в диалог будем назначать параметры цвета/шрифта которые заданы у соответствующей демонстрационной панели (**p_NA_record** или **p_A_record**). Зададим обработчики событий нажатия кнопок, реализующих возможности настройки отображения следующим образом:

```
procedure TForm1.B_NA_ColorSelClick(Sender: TObject);
begin
    // Заносим значение текущего цвета в диалог ColorDialog1
    ColorDialog1.Color := p_NA_record.Color;
    // Если запускается диалог выбора цвета
    if ColorDialog1.Execute then
        // то назначаем цвет панели примера - p_NA_record
        p_NA_record.Color := ColorDialog1.Color;
    // Обновление (перерисовка) DBGrid1
    DBGrid1.Invalidate;
end;

procedure TForm1.B_NA_FontSelClick(Sender: TObject);
begin
    // Заносим параметры текущего шрифта в диалог FontDialog1
    FontDialog1.Font := p_NA_record.Font;
    // Если запускается диалог выбора цвета
    if FontDialog1.Execute then
        // то назначаем шрифт панели примера - p_NA_record
        p_NA_record.Font := FontDialog1.Font;
    DBGrid1.Invalidate;
end;

procedure TForm1.B_A_ColorSelClick(Sender: TObject);
begin
    // Заносим значение текущего цвета в диалог ColorDialog1
    ColorDialog1.Color := p_A_record.Color;
    // Если запускается диалог выбора цвета
    if ColorDialog1.Execute then
        // то назначаем цвет панели примера - p_A_record
        p_A_record.Color := ColorDialog1.Color;
    DBGrid1.Invalidate;
end;

procedure TForm1.B_A_FontSelClick(Sender: TObject);
begin
    // Заносим параметры текущего шрифта в диалог FontDialog1
    FontDialog1.Font := p_A_record.Font;
    // Если запускается диалог выбора цвета
    if FontDialog1.Execute then
        // то назначаем шрифт панели примера - p_A_record
        p_A_record.Font := FontDialog1.Font;
    DBGrid1.Invalidate;
end;
```


Для того чтобы учитывались параметры подсветки записей которые задаются в панелях **p_NA_record** и **p_A_record** изменим обработчик события **OnDrawColumnCell** для компонента **DBGrid1**:

```
procedure TForm1.DBGrid1DrawColumnCell(Sender: TObject; const Rect:
  TRect; DataCol: Integer; Column: TColumn; State: TGridDrawState);
begin
  with DBGrid1.Canvas do
  begin // Проверим, не меньше ли значение поля sCount
    // чем значение установленное в SpinEdit1
    if (DataModule2.Q_v_storagesCount.value <= SpinEdit1.value) then
    // Если условие выполняется, то зададим формат вывода
    begin
      // Установим параметров шрифта и фона из панели p_NA_record
      Font := p_NA_record.Font;
      Brush.Color := p_NA_record.Color;
      if (gdSelected in State) then // если запись выбрана
      begin
        // Установим параметров шрифта и фона из панели p_A_record
        Font := p_A_record.Font;
        Brush.Color := p_A_record.Color;
      end;
    end;
  end; // With
  // Выполним отрисовку поля стандартным методом
  DBGrid1.DefaultDrawColumnCell(Rect, DataCol, Column, State);
end;
```

После изменения кода можно проверить, как работает приложение, задав во время работы иные оттенки цвета для фона и шрифта при подсветке записей. Пример функционирования приведен на рис 6.7

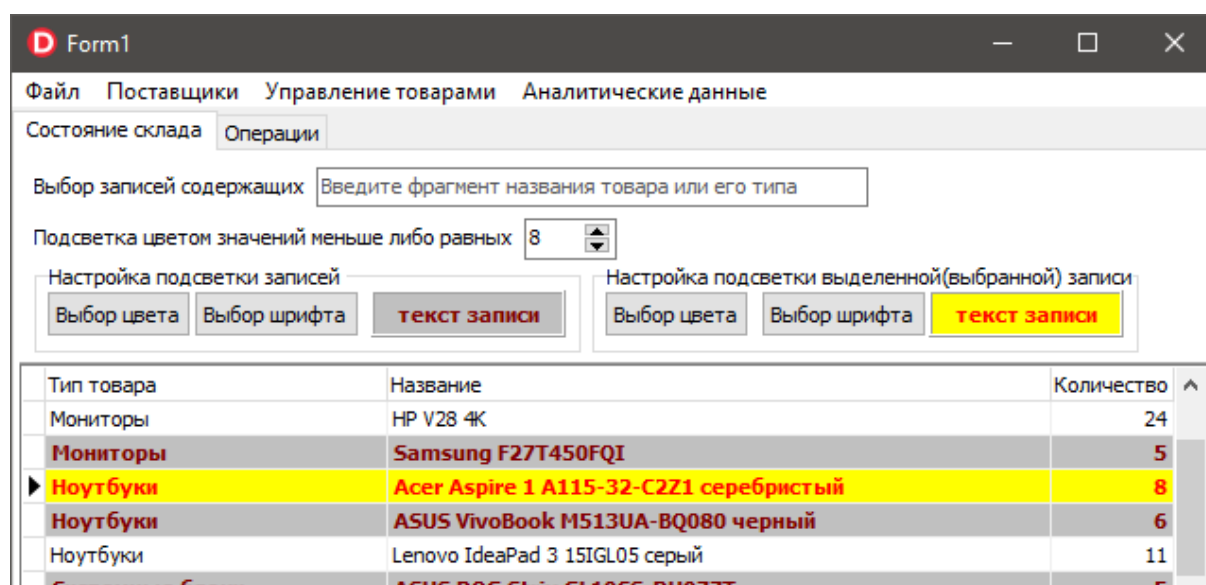


Рис 6.7 Пример работы измененной подсветки во время выполнения приложения

КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Каким образом назначаются изображения BLOB-полю?
2. Какие свойства компонентов **TImage** и **TDBImage** определяют параметры отображения изображения?
3. Для каких целей предназначен редактор столбцов?
4. Охарактеризуйте способы изменения местоположения столбца.
5. Каким образом можно изменить заголовки столбцов в компоненте **TDBGrid**?
6. Перечислите свойства столбцов, которые можно изменять программно.
7. Опишите процесс изменения сортировки полей во время выполнения приложения на основе изменения запроса.
8. Какие поля не могут быть использованы для сортировки?
9. Каким образом можно реализовать сортировку с использованием свойства **Sort**?
10. Каким образом осуществляется автоматическая прорисовка данных?
11. При помощи обработчика какого события можно настроить подсветку записей?
12. Как можно определить является ли запись выделенной?
13. Каким образом можно организовать изменение параметров шрифтов и цвета в момент работы приложения?